

LAPORAN TUGAS 2 (1 - 2)

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Yoanre Louis A.H.P (2410651039)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan sistem operasi modern menuntut pemahaman mendalam mengenai manajemen proses dan komunikasi antar proses (IPC) sebagai fondasi komputasi terkini. Praktikum ini mengimplementasikan dua paradigma kritis melalui sistem manajemen proses menggunakan teknik fork-exec-wait pattern dan aplikasi chat real-time dengan shared memory dan semaphore. Pada implementasi pertama, tiga child processes dieksekusi paralel untuk tugas berbeda—sorting array dengan bubble sort, linear search, dan statistical calculation—yang terkoordinasi melalui pipe communication dan sinkronisasi waitpid(). Pada implementasi kedua, aplikasi chat mengadopsi arsitektur client-server dengan shared memory sebagai media penyimpanan pesan terpusat dan semaphore binary untuk mengimplementasikan reader-writer lock pattern. Mekanisme sinkronisasi ini menjamin mutual exclusion pada critical section, mencegah race conditions ketika multiple processes mengakses shared memory secara bersamaan. Pemahaman praktis mengenai process lifecycle management, IPC mechanisms, dan concurrency control melalui implementasi langsung ini menjadi kompetensi esensial dalam pengembangan sistem terdistribusi dan aplikasi real-time yang memerlukan koordinasi antar proses yang robust dan efisien.

1.2 RUMUSAN MASALAH

1. Bagaimana implementasi manajemen proses hierarchical menggunakan teknik fork-exec-wait pattern untuk membuat tiga child processes yang melakukan tugas komputasi berbeda (sorting, searching, calculation) secara paralel dengan koordinasi melalui pipe communication?
2. Bagaimana mekanisme Interprocess Communication (IPC) dengan shared memory dan semaphore diimplementasikan untuk membangun aplikasi chat real-time yang mendukung multiple concurrent users dengan garansi data consistency?
3. Bagaimana efektivitas reader-writer lock pattern menggunakan semaphore dalam mencegah race condition dan menjamin mutual exclusion pada shared memory ketika multiple processes melakukan operasi baca-tulis secara bersamaan?

1.3 TUJUAN

1. Mengimplementasikan dan menganalisis sistem manajemen proses hierarchical melalui teknik fork-exec-wait pattern dengan tiga child processes terpisah yang melakukan tugas komputasi spesifik (sorting, searching, calculation) serta mengukur performa eksekusi menggunakan time measurement precision.
2. Membangun aplikasi chat berbasis IPC dengan memanfaatkan shared memory sebagai media penyimpanan pesan terpusat dan semaphore binary sebagai mekanisme sinkronisasi critical section yang mendukung multiple concurrent users dalam lingkungan real-time.
3. Mengevaluasi efektivitas mekanisme sinkronisasi reader-writer lock pattern menggunakan semaphore dalam menjamin mutual exclusion, mencegah race conditions, dan memastikan data consistency pada shared memory melalui testing aplikasi chat dengan multiple concurrent access scenarios.

BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

2.1.1 Sistem Manajemen Proses

Program sistem manajemen proses ini merepresentasikan implementasi konkurensi dan koordinasi antar proses dalam sistem operasi melalui teknik fork-exec-wait pattern. Kode tersebut menginisiasi tiga child processes yang dieksekusi paralel, masing-masing melakukan tugas komputasi berbeda: pengurutan array dengan algoritma bubble sort, pencarian elemen menggunakan linear search, serta kalkulasi statistik dasar seperti nilai rata-rata dan ekstrem. Parent process berfungsi sebagai coordinator yang mengatur eksekusi dengan membuat multiple pipes untuk komunikasi antar proses, mengumpulkan hasil komputasi, serta melakukan sinkronisasi menggunakan waitpid() untuk mencegah race conditions. Setiap proses child mengukur waktu eksekusi dengan precision microsecond melalui gettimeofday(), menyimpan hasil dalam struktur data terdefinisi, dan mengirimkannya kembali ke parent melalui mekanisme pipe IPC. Parent process kemudian mengkonsolidasi seluruh hasil termasuk PID, execution time, dan exit status untuk ditampilkan dalam format terstruktur, sekaligus menghitung total waktu eksekusi sistem. Implementasi ini mendemonstrasikan prinsip process isolation, resource sharing, dan inter-process communication yang menjadi fondasi sistem operasi modern, dengan optimasi melalui paralelisasi tugas untuk meningkatkan throughput sistem secara signifikan.

```
dany@LAPTOP-GRJL2V1D:~$ touch management_process.c
dany@LAPTOP-GRJL2V1D:~$ nano management_process.c
dany@LAPTOP-GRJL2V1D:~$ gcc -o management_process management_process.c
dany@LAPTOP-GRJL2V1D:~$ ./management_process
=== SISTEM MANAJEMEN PROSES SEDERHANA ===
Parent Process PID: 578

Process 579 (Sorting): Mengurutkan array...
Process 579 (Sorting): Array telah terurut dengan benar
Process 580 (Searching): Mencari nilai dalam array...
Process 580 (Searching): Nilai 42 ditemukan pada index 39

=== PARENT PROCESS MENUNGGU CHILD SELESAI ===
Process 581 (Calculation): Menghitung statistik...
Process 581 (Calculation): Sum=47684, Min=11, Max=996, Average=476.84

=== HASIL EKSEKUSI SEMUA PROSES ===


| PID | Task              | Time (s) | Status  |
|-----|-------------------|----------|---------|
| 579 | Sorting Array     | 0.000107 | Success |
| 580 | Searching Value   | 0.000062 | Success |
| 581 | Calculation Stats | 0.000067 | Success |


=== STATISTIK KESELURUHAN ===
Total processes: 3
Total execution time: 0.000236 seconds
Parent Process PID: 578
Semua child processes telah selesai dieksekusi!
dany@LAPTOP-GRJL2V1D:~$
```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

A. PROSES KOMPILASI:

- Preprocessing: File source code (process_manager.c) diproses oleh preprocessor untuk mengekspansi header files (#include) dan makro
- Compilation: Kode C dikompilasi menjadi assembly language oleh GCC compiler
- Assembly: Kode assembly diubah menjadi object code (binary)
- Linking: Object code di-link dengan library sistem (seperti libc) untuk menghasilkan executable file
- Output: Menghasilkan file executable management_process yang siap dijalankan

B. PROSES EKSEKUSI:

- Load Program: Sistem operasi memuat executable ke memory dan membuat process control block
- Parent Process Initialization: Process parent diinisialisasi dengan PID unik dan mengalokasikan resources
- Memory Allocation: Array dan struktur data dialokasikan dalam heap/stack memory
- Pipe Creation: Membuat multiple pipes untuk komunikasi IPC antar processes
- Fork Execution:
 - Parent melakukan system call fork() sebanyak 3 kali
 - Setiap fork membuat child process identik dengan duplikasi memory space
 - Child processes mendapatkan PID baru dan PPID merujuk ke parent

C. CHILD PROCESSES EXECUTION:

- Task Distribution: Setiap child process mengeksekusi tugas berbeda berdasarkan index:
 - Child 0: Sorting dengan bubble sort algorithm
 - Child 1: Searching dengan linear search
 - Child 2: Calculation statistik matematika
- Time Measurement: Setiap process menghitung execution time menggunakan gettimeofday()
- IPC Communication: Hasil komputasi dikirim ke parent melalui pipes

D. PARENT COORDINATION:

- Synchronization: Parent menunggu semua child selesai dengan waitpid()
- Data Collection: Mengumpulkan hasil dari semua pipes

- Result Processing: Mengkonsolidasi data dan menghitung total execution time
- Output Display: Menampilkan summary hasil eksekusi dalam format terstruktur

E. RESOURCE CLEANUP:

- Pipe Closure: Semua file descriptors pipes ditutup
- Memory Deallocation: Resources dibebaskan kembali ke sistem
- Process Termination: Semua processes exit dengan status code yang sesuai

F. KODE C:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <sys/time.h>

#include <time.h>

#include <string.h>


#define ARRAY_SIZE 100

#define NUM_PROCESSES 3


// Struktur untuk menyimpan hasil proses

typedef struct {

    pid_t pid;

    char task_name[50];

    double execution_time;

    int status;

    char result[100];
```

```
} ProcessResult;
```

```
// Fungsi untuk menghasilkan array acak
```

```
void generate_random_array(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        arr[i] = rand() % 1000;  
    }  
}
```

```
// Fungsi untuk mengukur waktu eksekusi
```

```
double get_current_time() {  
    struct timeval tv;  
    gettimeofday(&tv, NULL);  
    return tv.tv_sec + tv.tv_usec / 1000000.0;  
}
```

```
// Tugas 1: Sorting (Bubble Sort)
```

```
void task_sorting(int arr[], int size) {  
    printf("Process %d (Sorting): Mengurutkan array...\n", getpid());  
  
    for (int i = 0; i < size - 1; i++) {  
        for (int j = 0; j < size - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];
```

```

        arr[j] = arr[j + 1];

        arr[j + 1] = temp;
    }

}

}

```

```

// Verifikasi hasil sorting

```

```

int sorted = 1;

for (int i = 0; i < size - 1; i++) {

    if (arr[i] > arr[i + 1]) {

        sorted = 0;

        break;

    }

}

```

```

printf("Process %d (Sorting): Array %s terurut dengan benar\n",

        getpid(), sorted ? "telah" : "tidak");

}

```

```

// Tugas 2: Searching (Linear Search)

```

```

void task_searching(int arr[], int size) {

    printf("Process %d (Searching): Mencari nilai dalam array...\n", getpid());

    int target = 42; // Nilai yang dicari

```

```

int found = 0;

int position = -1;

for (int i = 0; i < size; i++) {

    if (arr[i] == target) {

        found = 1;

        position = i;

        break;

    }

}

if (found) {

    printf("Process %d (Searching): Nilai %d ditemukan pada index %d\n",

        getpid(), target, position);

} else {

    printf("Process %d (Searching): Nilai %d tidak ditemukan\n",

        getpid(), target);

}

}

// Tugas 3: Calculation (Menghitung statistik)

void task_calculation(int arr[], int size) {

    printf("Process %d (Calculation): Menghitung statistik...\n", getpid());

```

```

int sum = 0;

int min = arr[0];

int max = arr[0];


for (int i = 0; i < size; i++) {

    sum += arr[i];

    if (arr[i] < min) min = arr[i];

    if (arr[i] > max) max = arr[i];

}


double average = (double)sum / size;


printf("Process %d (Calculation): Sum=%d, Min=%d, Max=%d, Average=%.2f\n",

    getpid(), sum, min, max, average);

}


int main() {

    printf("=== SISTEM MANAJEMEN PROSES SEDERHANA ===\n");

    printf("Parent Process PID: %d\n\n", getpid());


    int array[ARRAY_SIZE];

    generate_random_array(array, ARRAY_SIZE);


    pid_t pids[NUM_PROCESSES];

```

```
int pipefd[NUM_PROCESSES][2];

ProcessResult results[NUM_PROCESSES];


// Buat pipes untuk komunikasi

for (int i = 0; i < NUM_PROCESSES; i++) {

    if (pipe(pipefd[i]) == -1) {

        perror("Pipe creation failed");

        exit(1);

    }

}


// Buat multiple child processes

for (int i = 0; i < NUM_PROCESSES; i++) {

    pids[i] = fork();

    if (pids[i] == -1) {

        perror("Fork failed");

        exit(1);

    }

    if (pids[i] == 0) {

        // Child process

        close(pipefd[i][0]); // Tutup read end
```

```

double start_time = get_current_time();

// Eksekusi tugas berdasarkan index
switch (i) {
    case 0:
        task_sorting(array, ARRAY_SIZE);
        strcpy(results[i].task_name, "Sorting Array");
        break;
    case 1:
        task_searching(array, ARRAY_SIZE);
        strcpy(results[i].task_name, "Searching Value");
        break;
    case 2:
        task_calculation(array, ARRAY_SIZE);
        strcpy(results[i].task_name, "Calculation Stats");
        break;
}

double end_time = get_current_time();

// Isi data hasil
results[i].pid = getpid();
results[i].execution_time = end_time - start_time;
results[i].status = 0; // Success

```

```

        // Kirim hasil ke parent melalui pipe

        write(pipefd[i][1], &results[i], sizeof(ProcessResult));

        close(pipefd[i][1]);

        exit(0);
    }
}

// Parent process - mengkoordinasi dan mengumpulkan hasil

printf("\n=== PARENT PROCESS MENUNGGU CHILD SELESAI ===\n");

for (int i = 0; i < NUM_PROCESSES; i++) {

    close(pipefd[i][1]); // Tutup write end di parent


    // Baca hasil dari pipe

    ProcessResult result;

    read(pipefd[i][0], &result, sizeof(ProcessResult));

    results[i] = result;

    close(pipefd[i][0]);


    // Tunggu child process selesai

    int status;

```

```

waitpid(pids[i], &status, 0);

if (WIFEXITED(status)) {
    results[i].status = WEXITSTATUS(status);
}
}

// Tampilkan hasil summary

printf("\n=== HASIL EKSEKUSI SEMUA PROSES ===\n");

printf("%-10s %-20s %-12s %-8s\n",
        "PID", "Task", "Time (s)", "Status");

printf("-----\n");

double total_time = 0;

for (int i = 0; i < NUM_PROCESSES; i++) {
    printf("%-10d %-20s %-12.6f %-8s\n",
            results[i].pid,
            results[i].task_name,
            results[i].execution_time,
            results[i].status == 0 ? "Success" : "Failed");

    total_time += results[i].execution_time;
}

printf("\n=== STATISTIK KESELURUHAN ===\n");

```

```
printf("Total processes: %d\n", NUM_PROCESSES);

printf("Total execution time: %.6f seconds\n", total_time);

printf("Parent Process PID: %d\n", getpid());

printf("Semua child processes telah selesai dieksekusi!\n");


return 0;

}
```

2.2 PRAKTIK TUGAS 2

2.2.1 Chat Application dengan IPC

Aplikasi chat ini mengimplementasikan sistem komunikasi inter-process (IPC) yang sophisticated menggunakan shared memory dan semaphore untuk sinkronisasi. Arsitekturnya terdiri dari tiga komponen utama: server sebagai pengelola resources, writer sebagai pengirim pesan, dan reader sebagai pembaca pesan. Shared memory berfungsi sebagai central repository yang menyimpan struktur data chat_room berisi history pesan, counter pesan, dan active reader count. Semaphore digunakan untuk mengimplementasikan reader-writer lock pattern yang menjamin konsistensi data dalam lingkungan konkurensi. Server menginisialisasi shared memory segment dan semaphore set, kemudian memantau status sistem secara real-time. Writer processes melakukan operasi write dengan terlebih dahulu mengakuisisi write semaphore untuk menjamin exclusive access, mencegah race conditions. Reader processes menggunakan mekanisme yang lebih kompleks dengan increment/decrement reader count dan appropriate semaphore operations untuk allow multiple concurrent reads while blocking writes. Implementasi ini menjamin data integrity melalui proper synchronization mechanisms yang mencegah corrupted states dan memastikan reliable message delivery antara multiple processes yang berjalan secara paralel.

- **Buat file chat.c:**

```
dany@LAPTOP-GRJL2V1D:~/louis$ nano server.c
dany@LAPTOP-GRJL2V1D:~/louis$ nano writer.c
dany@LAPTOP-GRJL2V1D:~/louis$ nano reader.c
dany@LAPTOP-GRJL2V1D:~/louis$ touch chatfile
dany@LAPTOP-GRJL2V1D:~/louis$ gcc -o server server.c
dany@LAPTOP-GRJL2V1D:~/louis$ ./server
=== CHAT SERVER STARTING ===
Using key: 1127279469
Shared Memory ID: 1
Semaphore ID: 1

=== CHAT SERVER STARTED ===
Server is running...
Press Ctrl+C to shutdown server

=== SERVER STATUS ===
Messages: 0 | Active Users: 0
=====
```

- **Terminal 2:**

```
dany@LAPTOP-GRJL2V1D:~$ cd louis
dany@LAPTOP-GRJL2V1D:~/louis$ gcc -o writer writer.c
dany@LAPTOP-GRJL2V1D:~/louis$ ./writer User1
Connecting to chat server...
Key: 1127279469
=== CHAT ROOM - Welcome User1 ===
Commands:
- Type messages to send
- 'history' to see all messages
- 'exit' to quit

User1> Hallo Louis
Message sent: User1> Hallo Louis
User1> Apa Kabar
Message sent: User1> Apa Kabar
User1> Hallo Louis
Message sent: User1> Hallo Louis
User1> exit
Goodbye User1!
dany@LAPTOP-GRJL2V1D:~/louis$
```

- **Terminal 3:**

```
dany@LAPTOP-GRJL2V1D:~$ cd louis
dany@LAPTOP-GRJL2V1D:~/louis$ ./writer User2
Connecting to chat server...
Key: 1127279469
=== CHAT ROOM - Welcome User2 ===
Commands:
- Type messages to send
- 'history' to see all messages
- 'exit' to quit

User2> Hallo juga
Message sent: User2> Hallo juga
User2> Wah Baik Nih
Message sent: User2> Wah Baik Nih
User2> exit
Goodbye User2!
dany@LAPTOP-GRJL2V1D:~/louis$
```

- **Terminal 4:**

```
dany@LAPTOP-GRJL2V1D:~$ cd louis
dany@LAPTOP-GRJL2V1D:~/louis$ ./reader
Trying to connect with key: 1127279469
=== CHAT ROOM READER ===
Type 'refresh' to see new messages, 'exit' to quit

Reader> refresh

=== NEW MESSAGES ===
User1> Hallo Louis
User2> Hallo juga
User1> Apa Kabar
User2> Wah Baik Nih
User1> Hallo Louis
=====

Reader> exit
Reader terminated.
dany@LAPTOP-GRJL2V1D:~/louis$
```

2.2.2 Penjelasan Proses Kompilasi dan Eksekusi:

A. PROSES KOMPILASI

- File Preparation: Membuat file `'chatfile'` menggunakan command `'touch'` sebagai key reference untuk fungsi `'ftok()'` dalam IPC
- Source Code Compilation: Mengkompilasi tiga file terpisah menggunakan GCC dengan command `'gcc -o [output] [source].c'`
- Preprocessing: Preprocessor mengekspansi header files (`'stdio.h'`, `'stdlib.h'`, `'string.h'`, `'sys/ipc.h'`, `'sys/shm.h'`, `'sys/sem.h'`) dan makro definisi
- Compilation Phase: GCC menerjemahkan kode C menjadi assembly language yang spesifik terhadap arsitektur processor
- Assembly Translation: Assembler mengubah kode assembly menjadi machine code dalam bentuk object file
- Linking Process: Linker menggabungkan object file dengan system libraries yang diperlukan menghasilkan executable binaries `'server'`, `'writer'`, dan `'reader'`

B. PROSES EKSEKUSI SERVER

- Server Initialization: Menjalankan `'./server'` untuk memulai chat server sebagai pengelola resources IPC
- IPC Key Generation: Membuat key IPC menggunakan `'ftok("chatfile", 'C')'` atau fallback key manual 1234
- Shared Memory Creation: Membuat shared memory segment dengan `'shmget()'` menggunakan flag `'IPC_CREAT | 0666'`
- Semaphore Setup: Membuat semaphore set dengan dua semaphores menggunakan `'semget()'` dengan konfigurasi yang sama
- Memory Initialization: Mengattach shared memory ke address space proses dengan `'shmat()'` dan menginisialisasi struktur `'chat_room'`
- Semaphore Configuration: Mengatur nilai awal semaphore menggunakan `'semctl()'` dengan operasi `'SETVAL'` (`mutex = 1`, `write_sem = 1`)
- Signal Handling: Mengimplementasi handler untuk `'SIGINT'` dan `'SIGTERM'` untuk cleanup resources secara graceful

C. PROSES EKSEKUSI CLIENT

- Client Launch: Menjalankan `./writer [username]` atau `./reader` dengan parameter username untuk identifikasi
- IPC Connection: Client processes melakukan lookup shared memory dan semaphore yang sudah dibuat server
- Memory Attachment: Mengattach existing shared memory segment ke address space proses client menggunakan `shmat()`
- Semaphore Access: Mendapatkan handle ke semaphore set yang sudah terinisialisasi menggunakan `semget()`
- User Authentication: Validasi koneksi dan memastikan server sedang berjalan sebelum memulai sesi chat

D. MESSAGE PROCESSING MECHANISM

- Input Handling: Menerima input pengguna melalui `fgets()` dan memproses special commands (`'exit'`, `'history'`, `'refresh'`)
- Critical Section Management: Menggunakan semaphore operations dengan `semop()` untuk mutual exclusion dalam reader-writer pattern
- Message Construction: Membuat formatted message dengan kombinasi username dan content text menggunakan `snprintf()`
- Shared Memory Access: Menyimpan pesan ke buffer messages dalam shared memory dengan proper synchronization
- Buffer Management: Mengimplementasi fixed-size array dengan `message_count` untuk tracking kapasitas

E. SYNCHRONIZATION PROTOCOL

- Writer Synchronization: Writer processes mengakuisisi write semaphore sebelum modifying shared memory untuk menjamin exclusive access
- Reader Synchronization: Reader processes menggunakan mekanisme reader-count dengan mutex protection untuk allow concurrent reads
- Race Condition Prevention: Semaphore operations menjamin hanya satu writer atau multiple readers yang dapat mengakses critical section

- Data Consistency: Mekanisme lock-unlock yang proper mencegah data corruption dan memastikan konsistensi chat history

F. INTERPROCESS COMMUNICATION

- Multi-Process Coordination: Multiple client processes dapat berjalan concurrently dengan username berbeda dalam shared environment
- Real-Time Message Broadcasting: Pesan yang dikirim oleh satu writer immediately available untuk semua processes melalui shared memory
- History Sharing: Semua processes dapat mengakses complete chat history melalui shared memory segment yang sama
- State Synchronization: Perubahan pada message_count dan reader_count secara real-time terlihat oleh semua connected processes

G. TERMINATION SEQUENCE

- Command Detection: Mendeteksi 'exit' command untuk menginisiasi proses termination sequence
- Resource Cleanup: Client melepas shared memory attachment menggunakan 'shmdt()' sebelum exit
- Server Cleanup: Server melakukan comprehensive cleanup dengan 'shmctl(IPC_RMID)' dan 'semctl(IPC_RMID)' pada termination
- Graceful Exit: Program menampilkan goodbye message dan melepas semua allocated resources secara proper
- Persistence Management: Shared memory tetap aktif selama server berjalan untuk mendukung client connections yang ongoing

H. KODE C

- Kode c untuk server.c:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/sem.h>
```

```
#include <signal.h>
```

```
#define SHM_SIZE 1024
```

```
#define MAX_MESSAGES 10
```

```
#define MESSAGE_LENGTH 100
```

```
typedef struct {
```

```
    char messages[MAX_MESSAGES][MESSAGE_LENGTH];
```

```
    int message_count;
```

```
    int reader_count;
```

```
} chat_room;
```

```
int shmid, semid;
```

```
chat_room *chat;
```

```
void sem_op(int semid, int sem_num, int op) {
```

```
    struct sembuf sb;
```

```
    sb.sem_num = sem_num;
```

```
    sb.sem_op = op;
```

```
    sb.sem_flg = 0;
```

```
semop(semid, &sb, 1);

}

void cleanup(int sig) {

    printf("\nCleaning up...\n");

    if (chat != NULL) {

        shmdt(chat);

    }

    if (shmid != -1) {

        shmctl(shmid, IPC_RMID, NULL);

        printf("Shared memory removed\n");

    }

    if (semid != -1) {

        semctl(semid, 0, IPC_RMID);

        printf("Semaphore removed\n");

    }

    printf("Server terminated.\n");

    exit(0);

}
```

```

int main() {

    // Buat file untuk key

    system("touch chatfile 2>/dev/null");


    // Create key

    key_t key = ftok("chatfile", 'C');

    if (key == -1) {

        printf("ftok failed, using default key 1234\n");

        key = 1234;

    }


    printf("=== CHAT SERVER STARTING ===\n");

    printf("Using key: %d\n", key);


    // Create shared memory

    shmid = shmget(key, sizeof(chat_room), IPC_CREAT | 0666);

    if (shmid == -1) {

        perror("shmget failed");

        exit(1);

    }

    printf("Shared Memory ID: %d\n", shmid);


    // Attach shared memory

    chat = shmat(shmid, NULL, 0);

```

```
if (chat == (void *)-1) {  
    perror("shmat failed");  
    exit(1);  
}  
  
// Initialize shared memory  
memset(chat, 0, sizeof(chat_room));  
chat->message_count = 0;  
chat->reader_count = 0;  
  
// Create semaphore  
semid = semget(key, 2, IPC_CREAT | 0666);  
if (semid == -1) {  
    perror("semget failed");  
    exit(1);  
}  
printf("Semaphore ID: %d\n", semid);  
  
// Initialize semaphores  
semctl(semid, 0, SETVAL, 1); // mutex = 1  
semctl(semid, 1, SETVAL, 1); // write_sem = 1  
  
// Signal handling  
signal(SIGINT, cleanup);
```

```

signal(SIGTERM, cleanup);

printf("\n=== CHAT SERVER STARTED ===\n");

printf("Server is running...\n");

printf("Press Ctrl+C to shutdown server\n\n");


// Server loop

while(1) {

    printf("=== SERVER STATUS ===\n");

    printf("Messages: %d | Active Users: %d\n", chat->message_count, chat-
>reader_count);

    printf("=====\n\n");

    sleep(3);

}

return 0;

}

```

- Kode c untuk writer.c:

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/ipc.h>

#include <sys/shm.h>

```

```

#include <sys/sem.h>

#define MAX_MESSAGES 10

#define MESSAGE_LENGTH 100

typedef struct {

    char messages[MAX_MESSAGES][MESSAGE_LENGTH];

    int message_count;

    int reader_count;

} chat_room;

int shmid, semid;

chat_room *chat;

void sem_op(int semid, int sem_num, int op) {

    struct sembuf sb;

    sb.sem_num = sem_num;

    sb.sem_op = op;

    sb.sem_flg = 0;

    semop(semid, &sb, 1);

}

int main(int argc, char *argv[]) {

    if (argc != 2) {

```

```
    printf("Usage: %s <username>\n", argv[0]);

    exit(1);
}

char *username = argv[1];

// Create key (sama dengan server)
key_t key = ftok("chatfile", 'C');
if (key == -1) {
    key = 1234;
}

printf("Connecting to chat server...\n");
printf("Key: %d\n", key);

// Get shared memory
shmid = shmget(key, sizeof(chat_room), 0666);
if (shmid == -1) {
    perror("Failed to access shared memory");
    printf("Please make sure the server is running first!\n");
    exit(1);
}

// Attach shared memory
```

```

chat = shmat(shmid, NULL, 0);

if (chat == (void *)-1) {

    perror("shmat failed");

    exit(1);

}


// Get semaphore

semid = semget(key, 2, 0666);

if (semid == -1) {

    perror("semget failed");

    exit(1);

}


printf("=== CHAT ROOM - Welcome %s ===\n", username);

printf("Commands:\n");

printf(" - Type messages to send\n");

printf(" - 'history' to see all messages\n");

printf(" - 'exit' to quit\n\n");


char input[80];


while(1) {

    printf("%s> ", username);

    fflush(stdout);

```

```
if (fgets(input, sizeof(input), stdin) == NULL) {  
    break;  
}
```

```
// Remove newline
```

```
input[strcspn(input, "\n")] = 0;
```

```
if (strcmp(input, "exit") == 0) {  
    break;  
}
```

```
else if (strcmp(input, "history") == 0) {
```

```
    // Read messages
```

```
    sem_op(semid, 0, -1);
```

```
    chat->reader_count++;
```

```
    if (chat->reader_count == 1) {
```

```
        sem_op(semid, 1, -1);
```

```
    }
```

```
    sem_op(semid, 0, 1);
```

```
    printf("\n=== CHAT HISTORY ===\n");
```

```
    if (chat->message_count == 0) {
```

```
        printf("No messages yet.\n");
```

```
    } else {
```

```

    for (int i = 0; i < chat->message_count; i++) {

        printf("%s\n", chat->messages[i]);

    }

}

printf("=====\n\n");

```

```

sem_op(semid, 0, -1);

chat->reader_count--;

if (chat->reader_count == 0) {

    sem_op(semid, 1, 1);

}

sem_op(semid, 0, 1);

}

else if (strlen(input) > 0) {

    // Send message

    sem_op(semid, 1, -1);


    if (chat->message_count < MAX_MESSAGES) {

        // Construct message manually

        char message[MESSAGE_LENGTH];

        snprintf(message, MESSAGE_LENGTH, "%s> %s", username, input);

        message[MESSAGE_LENGTH-1] = '\0';


        strcpy(chat->messages[chat->message_count], message);
    }
}

```

```

        chat->message_count++;

        printf("Message sent: %s\n", message);

    } else {

        printf("Chat room is full! Cannot send message.\n");

    }

    sem_op(semid, 1, 1);

}

}

shmdt(chat);

printf("Goodbye %s!\n", username);

return 0;

}

```

- Kode c untuk reader.c:

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/ipc.h>

#include <sys/shm.h>

#include <sys/sem.h>

```

```

#define MAX_MESSAGES 10

#define MESSAGE_LENGTH 100

typedef struct {

    char messages[MAX_MESSAGES][MESSAGE_LENGTH];

    int message_count;

    int reader_count;

} chat_room;

int shmid, semid;

chat_room *chat;

void sem_op(int semid, int sem_num, int op) {

    struct sembuf sb;

    sb.sem_num = sem_num;

    sb.sem_op = op;

    sb.sem_flg = 0;

    semop(semid, &sb, 1);

}

int main() {

    // Create key (sama dengan server)

    key_t key = ftok("chatfile", 'C');

    if (key == -1) {

```

```

    key = 1234;
}

printf("Connecting to chat server...\n");

printf("Key: %d\n", key);

// Get shared memory

shmid = shmget(key, sizeof(chat_room), 0666);

if (shmid == -1) {

    perror("Failed to access shared memory");

    printf("Please make sure the server is running first!\n");

    exit(1);

}

// Attach shared memory

chat = shmat(shmid, NULL, 0);

if (chat == (void *)-1) {

    perror("shmat failed");

    exit(1);

}

// Get semaphore

semid = semget(key, 2, 0666);

if (semid == -1) {

```

```
perror("semget failed");

exit(1);

}

printf("=== CHAT ROOM READER ===\n");

printf("Type 'refresh' to see new messages, 'exit' to quit\n\n");


int last_count = 0;

char input[10];


while(1) {

    printf("reader> ");

    fflush(stdout);


    if (fgets(input, sizeof(input), stdin) == NULL) {

        break;

    }


    input[strcspn(input, "\n")] = 0;


    if (strcmp(input, "exit") == 0) {

        break;

    }

}
```

```

// Read messages

sem_op(semid, 0, -1);

chat->reader_count++;

if (chat->reader_count == 1) {

    sem_op(semid, 1, -1);

}

sem_op(semid, 0, 1);


if (chat->message_count > last_count) {

    printf("\n=== NEW MESSAGES ===\n");

    for (int i = last_count; i < chat->message_count; i++) {

        printf("%s\n", chat->messages[i]);

    }

    printf("=====\n\n");

    last_count = chat->message_count;

} else {

    printf("No new messages.\n");

}


sem_op(semid, 0, -1);

chat->reader_count--;

if (chat->reader_count == 0) {

    sem_op(semid, 1, 1);

}

```

```
    sem_op(semid, 0, 1);  
}  
  
shmdt(chat);  
printf("Reader terminated.\n");  
return 0;  
}
```

BAB III

PENUTUP

3.1 KESIMPULAN

Berdasarkan implementasi kedua tugas sistem operasi ini, dapat disimpulkan bahwa manajemen proses dan IPC merupakan komponen fundamental dalam pengembangan sistem terdistribusi. Pada Tugas 1, teknik fork-exec-wait pattern berhasil mendemonstrasikan eksekusi paralel tiga child processes dengan tugas berbeda (sorting, searching, calculation) yang terkoordinasi melalui pipe communication, menunjukkan peningkatan efisiensi melalui konkurensi. Pada Tugas 2, aplikasi chat mengimplementasikan IPC sophisticated menggunakan shared memory sebagai central repository dan semaphore untuk sinkronisasi reader-writer pattern, menjamin data consistency dalam lingkungan multiprocessing. Kedua implementasi membuktikan efektivitas mekanisme sinkronisasi dalam mencegah race conditions dan memastikan reliable communication antar processes. Pemahaman praktis ini memberikan fondasi kuat untuk pengembangan sistem operasi modern, aplikasi terdistribusi, dan arsitektur microservices yang memerlukan koordinasi antar proses yang robust dan efisien dalam menangani tugas-tugas komputasi yang kompleks.